

Pod Execution Model

How pods ship working software as fast as possible

02

Why Pods Exist

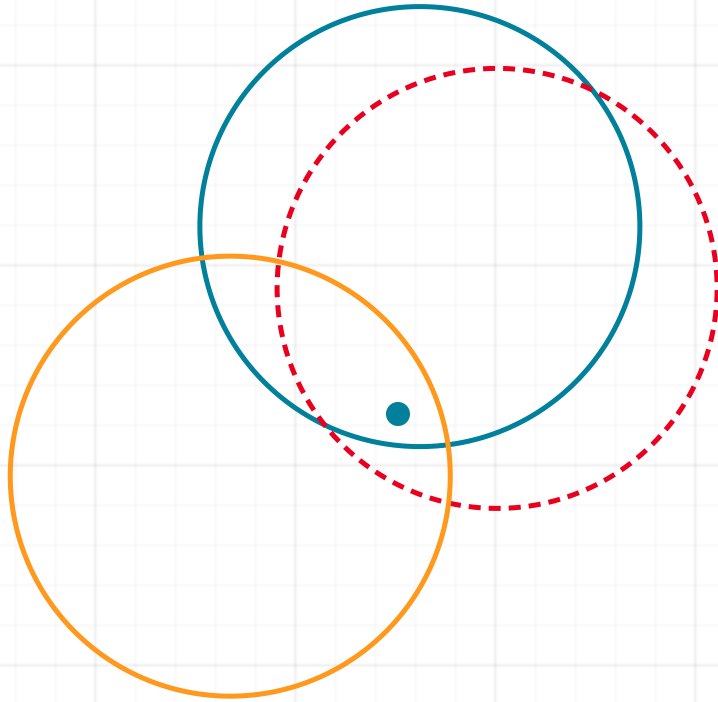
Pods compress delivery from months to weeks by replacing spec-driven work with prototype-first iteration.

- Small cross-functional strike teams
- Prototype first, not spec first
- Collapse roles, remove translation layers
- AI fills the gaps that used to require more people

Calendar ships fast. That proves the model. Then it spreads.

Continuous Iteration

Every delivery strengthens the next. The system learns as it runs.



01 Discover

Synthesize stakeholder signals, system context, and competitive inputs to define a differentiated solution direction.

DELIVERABLE: SIGNED-OFF PRD + PROMPT PACKAGE

02 Iterate

Iterate through working Canvas-driven prototypes with same-day feedback and pre-code stakeholder alignment. Then build, integrate, and continuously test against acceptance criteria.

DELIVERABLE: VALIDATED PROTOTYPE + WORKING BUILD IN STAGING

03 Launch

Ship to production users, run retrospective, capture learnings, and feed evidence back into the next Discover cycle.

DELIVERABLE: RELEASE + RETRO INTELLIGENCE REPORT

Execution becomes predictable when learning never stops.

04

Pod Structure

Tiny strike team. No large teams. No handoffs. No translation layers.

⚙ Engineer

🎨 Design

💻 Platform

✅ QA

📄 PM

🌐 Host

💖 CS (optional)

- Everyone prototypes. Everyone QA's. Everyone ships.
- AI is the force multiplier. Every person operates at 3-5x output.

05

Inputs Replace Deliverables

The old model passed documents between roles. The new model uses AI to generate working inputs into a shared prototype.

The Old Way

- ✗ PM writes PRD over days, hands to Design
- ✗ Designer spends weeks in Figma, hands to Eng
- ✗ Engineer builds from spec, hands to QA
- ✗ Each step adds weeks and loses context

The New Way: Roles Collapse

- ✓ Anyone with VS Code + an approved model (Opus 4.6, Codex, Claude 5.4) can draft, design, or build
- ✓ PM drafts the problem frame AND builds a clickable prototype
- ✓ Designer drafts the PRD AND ships working HTML
- ✓ Engineer scaffolds the app while QA agents, kAutomate, and automation generate structured validation coverage
- ✓ No handoffs. Everyone contributes everywhere.

AI is why 3 people can do what 10 used to. The next four slides show exactly how.

Industry Inputs

Feed AI all available context: customer signals, competitive products, internal presentations, articles, and domain expertise. No multi-week spec cycles.

Gather Context

- Feed meeting transcriptions directly into ChatGPT
- Paste Teams conversations, support tickets, customer interviews
- Output: structured summary of what's broken and why it matters

[ChatGPT](#)[Teams](#)[Copilot](#)

Draft MVP PRD

- Ask AI to generate a minimal PRD from all gathered context
- Scope to one core user flow. Cut everything else.
- Output: 1-page PRD with problem, scope, constraints, and "done" definition

[ChatGPT](#)[Copilot](#)

Build the Prototype

- PM opens VS Code + an approved model and builds a clickable prototype same day
- No designer needed for v1. Ship something testable immediately
- Output: working HTML the pod can react to, not a document to review

[VS Code](#)[Opus 4.6](#)[Codex](#)

What used to take weeks of manual synthesis now takes a morning with AI. Gather context, draft the PRD, move on.

07

Designer Inputs

UI/UX accelerates refinement after a working iteration, not by defining pixel-perfect specs upfront. Build clickable prototypes in a couple hours, not weeks.

Shape the Brief

- Take the PM's draft PRD and pressure-test it with UX thinking
- Feed user research and competitor screenshots into ChatGPT to surface gaps
- Output: refined brief with interaction requirements the PM might miss

ChatGPT

Copilot

Build in Code

- Open VS Code + an approved model. Describe the UI in plain language
- Full page with states, navigation, real copy. Working prototype, not static mockups
- Output: clickable prototype the pod can test same day

VS Code

Opus 4.6

Codex

Iterate Live

- Tell AI "make the sidebar narrower" or "add dark mode." See it instantly
- Refine the prototype alongside the engineer in the same codebase
- Designer becomes a creative director of AI output, not a pixel pusher

VS Code

Opus 4.6

| A designer with VS Code + an approved model can produce a clickable prototype in 2 hours.

Engineer Inputs

VS Code + an approved model scaffolds the app. Engineer directs, integrates, and ensures quality.

Scaffold the Real App

- Take prototypes from PM and designer. Use AI as a pair partner, pointed at our codebase.
- Scaffold production code from prototype patterns. Bridge the gap from demo to real
- Primary user flow running end-to-end in the real app within 48 hours

VS Code

Opus 4.6

Codex

Ship and Iterate

- "Add form validation" / "add offline support." Done in minutes, not sprints
- AI generates tests, API contracts, data models, and component variations
- Engineer's role shifts from writing code to directing AI and ensuring quality

VS Code

Opus 4.6

Codex

Own the Architecture

- Map integration surfaces: APIs, auth, data sources
- Use AI as a peer reviewer: surface risks, challenge assumptions, catch blind spots
- Deployable structure from day 1: CI, env config, feature flags

ChatGPT

VS Code

What used to take a sprint takes 2-3 days. Engineers orchestrate AI while continuous feedback replaces translation cycles and drives rapid iteration.

Stakeholder Inputs

QA is not a phase. It starts when the first prototype exists. Stakeholders remove friction, not approve output.

Embedded from Day 1

- QA joins at pod kickoff, not after engineering is "done"
- Tests the first prototype on day 1. Shapes requirements through what breaks
- Validate on prototypes, not specifications. Bugs caught in context, not after handoff
- QA supports pods, doesn't gate releases
- Uses AI to generate test scenarios, edge cases, and regression checklists from the evolving prototype

ChatGPT

VS Code

Everyone is QA

- PM validates assumptions against real user behavior, not specs
- Designer tests interactions in the working prototype, not in Figma comments
- Engineer uses AI to generate automated tests alongside every feature

VS Code

Opus 4.6

Stakeholder Alignment

- Strategic framing: why this matters right now
- Fast yes/no on scope questions. No committees
- Remove organizational friction. Clear the path so the pod can ship

There is no "hand off to QA." QA is already there. Everyone tests. Stakeholders clear the path.

Artifact Expectations

Every artifact should accelerate testing or iteration. If it doesn't, cut it.

What Pods Produce

- ✓ 1-page MVP PRD (problem, scope, done definition)
- ✓ Clickable prototypes (not static mockups)
- ✓ AI-generated test checklists and edge cases
- ✓ Working weekly demo for stakeholder feedback
- ✓ Short decision notes ("we chose X because Y")

What Pods Don't Produce

- ✗ Multi-page spec documents
- ✗ Polished Figma handoff files
- ✗ Status decks or approval presentations
- ✗ Documentation before the thing works
- ✗ Anything that requires a meeting to review

The artifact is the prototype. Everything else is a note that helps the prototype get better.

11

Iteration Speed

Continuous operational progress beats delayed polish.

- **Visible progress:** The working prototype advances continuously
- **Stakeholder reaction:** Feedback happens inside the real system, not in review meetings
- **Cross-pod learning:** Successful patterns spread immediately across teams

| Execution is the strategy. The pod that ships fastest teaches the org the most.

Definition of Done

Done means operational, testable, usable. Not polished, finalized, documented.

Done Means

- ✓ Operational: works end to end
- ✓ Testable: someone can try it
- ✓ Adoptable: user completes the core flow
- ✓ Demonstrable: showable in a working demo

Done Does Not Mean

- ✗ Pixel perfect
- ✗ Fully documented
- ✗ All edge cases handled
- ✗ Production hardened

| Pods ship capability, not perfection.

**"Teams no longer operate around a PRD.
They operate around a Prototype."**

Three Pilot Pods

One execution model. Three proving environments. Each validates a different adoption surface.

Calendar Pod

Steve · Damion · Denny

- Ship Calendar as a proving surface for rapid execution on real workflows
- Example pattern: Calendar for Submittals, Calendar for Inspections, Calendar for Closeout
- Learn what inputs are actually required to iterate fast
- This is the proving ground. If it works, the model expands org-wide after May release
- Establishes the template for how every future team operates

Strategic Apps Pod

Brian · Sakiba · Vlado

- Strategic execution on AssetCentric Project Management and expansion surfaces around it
- Two app docs already exist as starting material for immediate execution
- Already a high-performing team. Redirect into the faster input-driven execution model
- Proves the model works for strategic platform evolution, not just greenfield builds

Forward Deployed Pod

Jordan Hoff + engineer (TBD) + designer (TBD)

- Embed with owner customers. Visit, interview, understand the work first
- Identify high-value business processes that can be transformed with AI
- Discovery inside real workflows followed by immediate implementation
- Real-world customer process transformation, not internal prototyping exercises

Small teams, fast inputs, shared QA. Deliver in execution cycles, not planning cycles. Strategic Capital Planning pod operates until first customer-aligned execution pod launches.

Timeline

Definition to deployment in rapid cycles. Calendar is the pilot.

- **This Week:** Staff the Forward Deployed pod (engineer + designer for Jordan). Confirm all three pod rosters
- **Next Week:** Education and alignment meeting with everyone in all three pods. Calendar pod kicks off immediately after
- **As Fast As Possible:** Calendar pod ships working product. Model validated. Team can articulate what inputs are actually needed
- **After May Release:** Broader rollout begins pending validation from initial three pods. Calendar pod becomes the template every new team follows

What Changes Now

Concrete behavior shifts starting immediately.

- No more weeks of polish before learning. Ship unfinished but operational
- PMs prototype the same day they write the PRD. Problem frames become working artifacts
- Designers move into VS Code. Layout ships as working HTML, not Figma files
- Engineers direct AI, not write every line. Scaffolding, iteration, testing all accelerated
- QA is everyone's job, embedded from day 1. No downstream handoff
- Weeks, not months. New muscle memory starts now

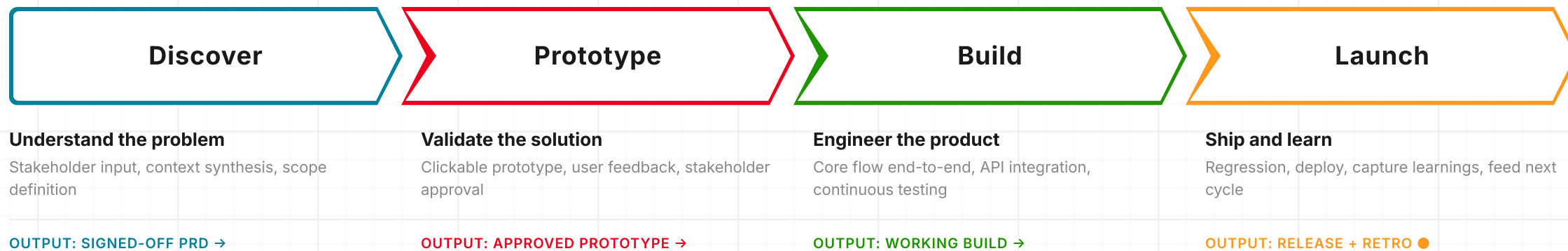
APPENDIX

Pod Execution System

18

The Flow

From problem to product. Four phases. No fixed timeline. Scale to fit.



Each phase has a clear output. The cycle runs in days or months. Same discipline.

Who Drives Each Phase

Every role is engaged every phase. Bold = peak ownership. The flow, not the calendar, determines who leads.

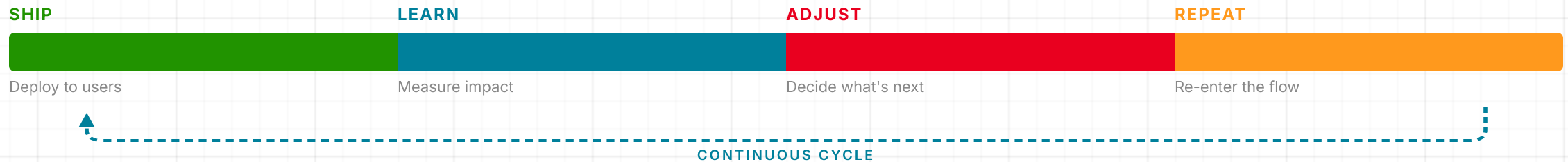
	DISCOVER	PROTOTYPE	BUILD	LAUNCH
PM	Meetings, synthesize PRD, lock scope	Build clickable prototype, validate with users	Refine scope, cut non-core, support testing	Soft launch coordination, feedback capture
Designer	Research, competitor audit, pattern library	Refine prototype, add all states and flows	Polish interactions directly in codebase	Final interaction polish, launch visuals
Engineer	Tech assessment, integration mapping, env setup	Scaffold real app from prototype patterns	Core flow end-to-end, API integration, testing	Bug fixes, edge cases, deploy, release
QA	Define test strategy from PRD	Test prototype, shape requirements through bugs	Continuous testing on real builds, edge cases	Regression pass, release sign-off
Pod Lead	Scope lock, define done, set cadence	Stakeholder demo, capture feedback, hold scope	Mid-cycle check, kill scope creep, unblock	Release decision, retrospective, capture lessons

This is not waterfall. Roles overlap. Phases compress. The grid shows peak ownership, not sequential handoffs.

20

Iteration Mechanisms

The system doesn't end at launch. Every cycle feeds the next.



After every release

Retro within 48 hours. What shipped, what slipped, what we learned.

Continuous feedback

Usage data flows directly into the next cycle. No separate research sprint.

Compounding speed

Patterns become reusable. The pod gets faster. Each cycle builds on the last.

Speed is earned, not assumed. The system compounds.

PM: Old Way vs New Way

From spec author to decision scientist. The PM's value is judgment, not documentation.

The Old Way

- ✗ Spends weeks writing detailed PRDs before anyone builds
- ✗ Gathers requirements through meetings and documents them in slides
- ✗ Hands finished spec to design, waits for mockups, hands to engineering
- ✗ Success measured by spec completeness, not product learning
- ✗ Manages timelines and Jira tickets, not outcomes

The New Way

- ✓ Defines problem frame inside a live prototype on day one
- ✓ Feeds stakeholder signals, competitive context, and customer data directly into AI to synthesize direction
- ✓ Builds a working clickable prototype same day using Canvas and approved models
- ✓ Validates assumptions through real user interaction, not spec review
- ✓ Operates as a decision scientist: chooses which problems to solve, not how to document them

| The PM who ships a prototype on Monday learns more than the PM who ships a PRD on Friday.

Designer: Old Way vs New Way

From pixel pusher to creative director of AI output. Design accelerates iteration, not precedes it.

The Old Way

- ✗ Spends weeks in Figma creating pixel-perfect mockups nobody has tested
- ✗ Delivers static screens with redline annotations for engineering to interpret
- ✗ Design-to-code handoff creates translation loss and rework cycles
- ✗ Feedback happens in comments on static files, disconnected from real behavior
- ✗ Waits for PRD before starting any visual work

The New Way

- ✓ Opens VS Code + an approved model and describes UI in plain language
- ✓ Produces working interactive prototypes in hours, testing real states and navigation
- ✓ Directs and curates AI-generated output like a creative director, not a production artist
- ✓ Iterates live alongside engineering in the same codebase, no handoff
- ✓ Refines after a working iteration exists, not before one is built

| A designer with an AI pair can produce what used to take a team of three two weeks, in a single afternoon.

Engineer: Old Way vs New Way

From line-by-line coder to AI orchestrator. The engineer's value is architecture and judgment, not keystrokes.

The Old Way

- ✗ Waits for spec and mockups before writing a single line of code
- ✗ Builds from scratch, manually writing boilerplate, tests, and API contracts
- ✗ Sprints measured by story points and velocity, not working product
- ✗ Code review is the only quality gate, happens after completion
- ✗ Integration and deployment are separate phases with separate teams

The New Way

- ✓ Takes prototype from PM/Designer and scaffolds production code from patterns on day one
- ✓ Uses AI as a pair partner: generates tests, API contracts, data models, and component variations
- ✓ Primary user flow running end-to-end in the real app within 48 hours
- ✓ Continuous feedback replaces translation cycles. Iteration happens in minutes, not sprints
- ✓ Owns architecture, integration surfaces, and deployment from day one with CI, env config, and feature flags

| The engineer who directs AI and ships in days outpaces the team that hand-codes for weeks.

QA: Old Way vs New Way

From post-delivery gatekeeper to embedded quality partner. Validation is continuous, not sequential.

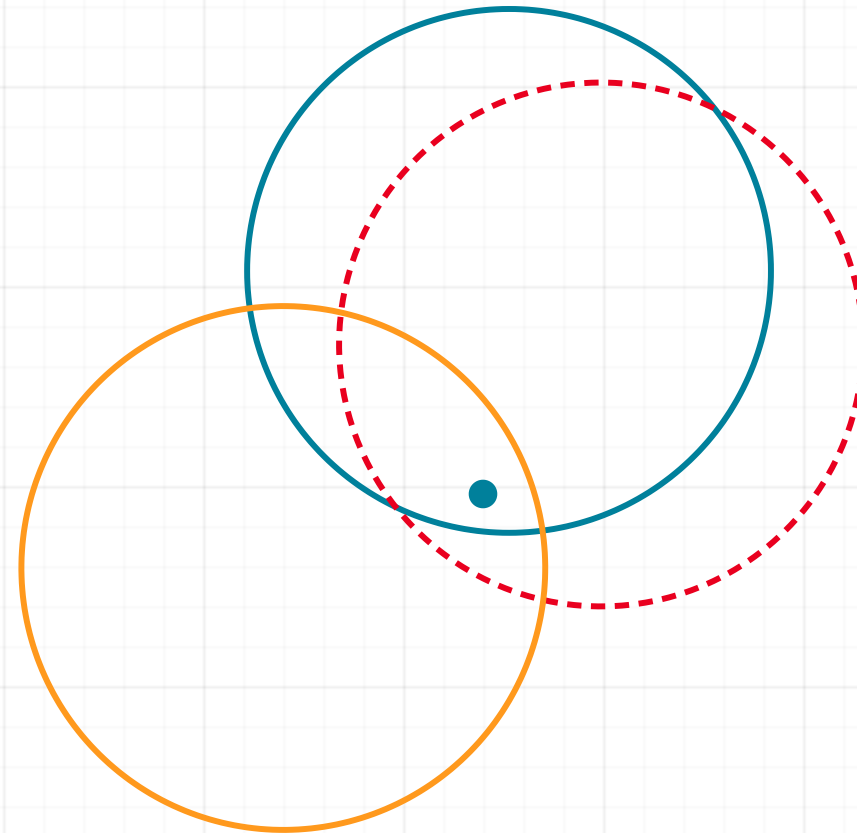
The Old Way

- ✗ Tests after engineering declares "done." Finds bugs after context is lost
- ✗ Writes manual test plans from static specifications
- ✗ QA is a phase: gated, scheduled, and siloed from development
- ✗ Regression testing is manual, slow, and incomplete
- ✗ "Hand off to QA" creates adversarial dynamic and delays release

The New Way

- ✓ Joins at pod kickoff. Tests the first prototype on day one
- ✓ QA agents, kAutomate, and AI generate test scenarios, edge cases, and regression checklists continuously
- ✓ Everyone is QA: PM validates assumptions, Designer tests interactions, Engineer generates automated tests
- ✓ Validates on working prototypes, not specifications. Bugs caught in context, not after handoff
- ✓ QA supports pods and accelerates release, never gates it

| There is no "hand off to QA." QA is already there. Validation is continuous, not a checkpoint.



CURRENT DRAFT

March 23, 2026